

INTELIGENCIA DE NEGOCIOS

Implementacion del ETL y construccion del DataMart

Caso: Colocaciones de Creditos Mivivienda 2024

Avance	Avance 2
Integrantes	Daniel Alonso Correa Chanta Jose Carlo Castro Franco
Universidad	Universidad Tecnologica del Peru
Curso	Inteligencia de Negocios
Docente	Oscar Eduardo Balcazar Chumacero
Lugar y fecha	Piura, Peru - 2026

Contenido

N.o	Seccion
1	Introduccion y objetivo del avance
2	Fuente de datos y perfil de calidad
3	Arquitectura y herramientas utilizadas
4	Implementacion fisica de la base de datos
5	Desarrollo del proceso ETL
6	Carga inicial e incremental
7	Validacion, pruebas y calidad
8	Dashboard web y consumo mediante APIs
9	Resultados de negocio y KPIs
10	Evidencias tecnicas
11	Conclusiones
-	Referencias
A	Anexo: explicacion de archivos Python y comandos

Resumen ejecutivo

El presente Avance 2 implementa fisicamente el DataMart definido en el Avance 1 para analizar las colocaciones de creditos Mivivienda durante 2024. La solucion utiliza Python para el proceso ETL, MySQL para staging y modelado dimensional, y Flask con HTML, CSS y JavaScript para la explotacion visual mediante un dashboard web.

El archivo origen contiene 13,507 filas. El proceso de calidad elimino 4,160 filas completamente vacias y 11 duplicados exactos, obteniendose 9,336 registros validos. La carga inicial inserto los 9,336 hechos y la ejecucion incremental posterior inserto cero registros, demostrando que la clave tecnica record_hash evita la duplicacion.

Resultado principal: DataMart operativo con cinco dimensiones, una tabla de hechos, carga incremental, validaciones sin errores y dashboard conectado directamente a la vista analitica de MySQL.

1. Introduccion y objetivo del avance

El Avance 1 definio el problema de negocio, los requerimientos analiticos, el diccionario inicial y un modelo estrella conceptual. Este segundo avance transforma esa propuesta en una solucion funcional, reproducible y verificable.

1.1 Objetivo

Desarrollar la integracion, transformacion y carga de datos para construir fisicamente un DataMart de colocaciones Mivivienda, incorporando tablas, relaciones, indices, restricciones, carga inicial, carga incremental, pruebas de integridad y evidencias de explotacion de informacion.

1.2 Alcance implementado

- Base de datos MySQL con staging, dimensiones, hechos y vistas.

- Proceso ETL desarrollado en Python y pandas.
- Limpieza, conversion de tipos, normalizacion y deduplicacion.
- Carga inicial e incremental mediante una clave SHA-256.
- Pruebas SQL, pruebas unitarias y pruebas de APIs.
- Backend Flask con APIs REST y frontend HTML, CSS y JavaScript.
- Dashboard con KPIs, graficos y filtros interactivos.

2. Fuente de datos y perfil de calidad

La fuente es el archivo CSV Colocaciones de credito Mivivienda_2024, publicado por Fondo MIVIVIENDA S.A. El archivo esta separado por punto y coma y contiene 14 columnas relacionadas con fecha, producto, ubicacion, institucion financiera, montos, plazo y tasa.

Indicador	Resultado	Tratamiento ETL
Filas fisicas	13,507	Lectura completa
Filas vacias	4,160	Eliminadas
Filas no vacias	9,347	Evaluadas
Duplicados exactos	11	Eliminados
Filas finales	9,336	Cargadas
Columnas	14	Esquema validado

2.1 Reglas de calidad aplicadas

- Las filas completamente vacias se eliminan antes de convertir tipos.
- Las fechas numericas aaaammdd se convierten a DATE.
- El ubigeo se conserva como texto CHAR(6).
- Los textos se limpian con TRIM, espacios uniformes y mayusculas.
- Los montos, tasas y plazos se convierten a valores numericos.
- Se rechazan montos, tasas y plazos menores o iguales a cero.
- Los duplicados exactos se eliminan antes de la carga.

3. Arquitectura y herramientas utilizadas

La solucion se implemento dentro de un solo proyecto. Python ejecuta el ETL y tambien funciona como backend web mediante Flask. MySQL almacena el DataMart y el navegador consume APIs JSON para construir la interfaz con JavaScript y Chart.js.

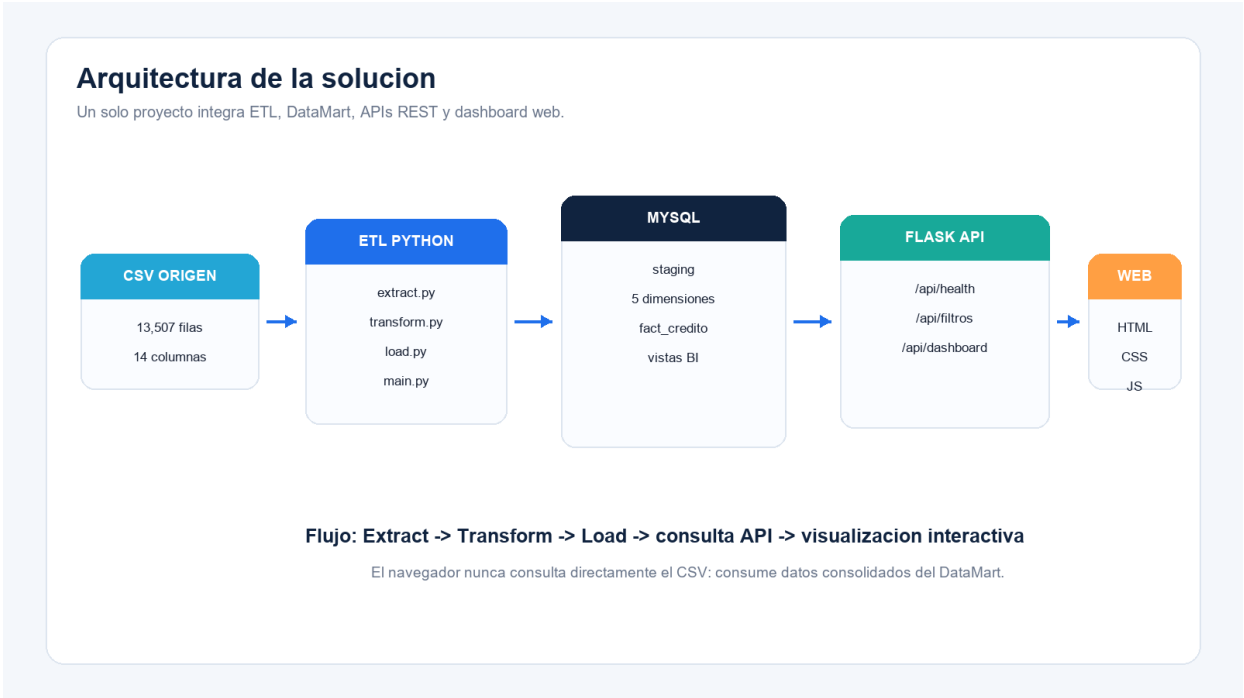


Figura 1. Arquitectura implementada para el Avance 2.

Componente	Tecnologia	Responsabilidad
Fuente	CSV	Datos de colocaciones 2024
ETL	Python, pandas	Extraer, limpiar, transformar y cargar
Persistencia	MySQL	Staging, DataMart, indices y restricciones
Backend	Flask, SQLAlchemy	APIs REST y consultas parametrizadas
Frontend	HTML, CSS, JavaScript	Interfaz, filtros y tabla
Graficos	Chart.js	Visualizaciones interactivas
Pruebas	unittest y SQL	Calidad, integridad y APIs

4. Implementacion fisica de la base de datos

La base configurada es `mivivienda_dm`. El script `setup_database.py` crea la base y ejecuta los archivos SQL de staging y DataMart. La tabla de hechos utiliza claves subrogadas hacia cinco dimensiones.

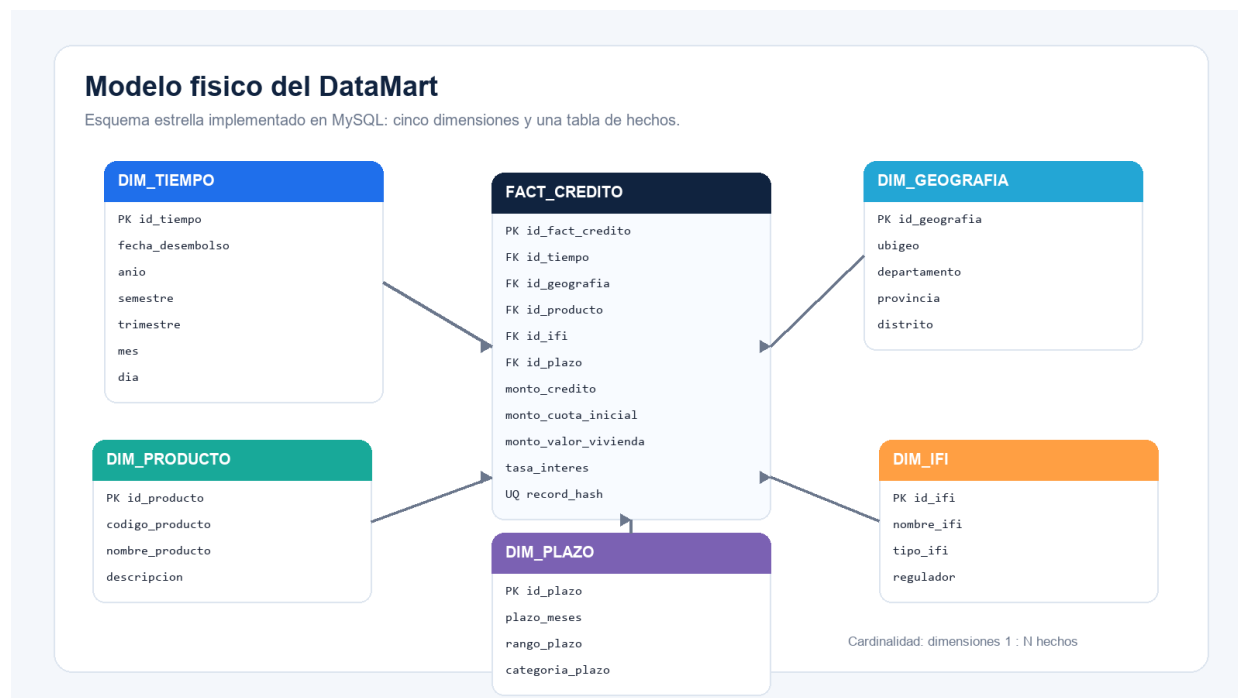


Figura 2. Modelo estrella físico implementado en MySQL.

4.1 Tablas implementadas

Tabla	Tipo	Descripción
stg_colocaciones_mivivienda	Staging	Area temporal del lote limpio
dim_tiempo	Dimension	Jerarquía de fecha, año, trimestre y mes
dim_geografia	Dimension	Ubigeo, departamento, provincia y distrito
dim_producto	Dimension	Código y descripción del producto
dim_ifi	Dimension	Institución y tipo de IFI
dim_plazo	Dimension	Meses, rango y categoría de plazo
fact_credito	Hechos	Medidas de cada crédito desembolsado
etl_ejecucion	Auditoria	Métricas de cada ejecución ETL

4.2 Relaciones, índices y restricciones

- Cinco claves foráneas relacionan fact_credito con las dimensiones.
- Las claves primarias son autoincrementales y subrogadas.
- record_hash posee una restricción UNIQUE para evitar duplicados.
- Se crearon índices sobre tiempo, geografía, producto, IFI, plazo y fecha de corte.
- Las restricciones CHECK validan montos, tasas, plazos y cantidad_credits.
- Las columnas críticas utilizan NOT NULL para asegurar integridad.

5. Desarrollo del proceso ETL

El ETL está dividido en módulos para separar responsabilidades. main.py funciona como orquestador y ejecuta Extract, Transform y Load en el orden correcto.

5.1 Extract

extract.py lee el CSV usando separador punto y coma, conserva inicialmente todas las columnas como texto y comprueba que el archivo posea exactamente las 14 columnas esperadas. La opción skip_blank_lines=False permite contar las filas vacías como evidencia.

```
df = pd.read_csv(  
    path,  
    sep=";",  
    dtype="string",  
    encoding="utf-8-sig",  
    skip_blank_lines=False,  
)
```

5.2 Transform

transform.py aplica limpieza, normalización, conversión de tipos, reglas de negocio y deduplicación. Al finalizar genera record_hash mediante SHA-256 con los 14 valores normalizados.

Clave técnica: La fuente no incluye un identificador único de transacción. record_hash representa el contenido completo de cada fila y permite detectar si un crédito ya fue cargado.

5.3 Load

load.py carga primero staging, después cada dimensión y finalmente fact_credito. Las dimensiones utilizan INSERT IGNORE y claves únicas. La tabla de hechos se carga mediante JOIN entre staging y las cinco dimensiones.

```
staging -> dim_tiempo  
         -> dim_geografia  
         -> dim_producto  
         -> dim_ifi  
         -> dim_plazo  
         -> fact_credito
```

5.4 Orquestación

```
df_raw = extract_colocaciones(source_path)  
df_clean, metrics = transform(df_raw)  
load_staging(df_clean, source_path)  
inserted_rows = load_datamart()  
record_execution(mode, source_path, metrics, inserted_rows)
```

6. Carga inicial e incremental

La carga inicial reinicia las dimensiones y la tabla de hechos para construir el DataMart desde cero. La carga incremental conserva los datos existentes y solo inserta hashes nuevos.

Evidencia de ejecucion ETL

Comparacion de carga inicial e incremental registrada en MySQL.

```
PowerShell - ETL Mivivienda

> python -m etl.main --mode initial

=== INICIO ETL MIVIVIENDA (INITIAL) ===

[EXTRACT] Filas leidas: 13,507

[TRANSFORM] Vacias: 4,160 | Duplicadas: 11 | Finales: 9,336

[LOAD] Hechos nuevos insertados: 9,336

=== ETL FINALIZADO - ESTADO OK ===

> python -m etl.main --mode incremental

=== INICIO ETL MIVIVIENDA (INCREMENTAL) ===

[EXTRACT] Filas leidas: 13,507

[TRANSFORM] Vacias: 4,160 | Duplicadas: 11 | Finales: 9,336

[LOAD] Hechos nuevos insertados: 0

=== ETL FINALIZADO - ESTADO OK ===
```

Figura 3. Evidencia de carga inicial e incremental.

Modo	Transformadas	Insertadas	Resultado
Initial	9,336	9,336	Carga completa
Incremental	9,336	0	Sin duplicacion

El resultado incremental de cero filas no significa que el proceso haya fallado. Significa que todos los registros del archivo ya existian en fact_credito y fueron correctamente identificados.

7. Validacion, pruebas y calidad

Las pruebas combinaron consultas SQL de integridad, conciliacion de conteos, restricciones fisicas y pruebas automatizadas de Python.



Figura 4. Conteos e indicadores de integridad del DataMart.

7.1 Pruebas automatizadas



Figura 5. Resultado de pruebas unitarias y de integracion.

Prueba	Resultado esperado	Resultado
Eliminacion de fila vacia	Fila descartada	Correcto
Eliminacion de duplicado	Una sola fila	Correcto
Monto negativo	Fila invalida	Correcto
Conexion API-MySQL	HTTP 200	Correcto
Filtro LIMA + NMIV	Solo datos filtrados	Correcto
Pagina principal	HTML disponible	Correcto

7.2 Conciliación con el origen

Concepto	Calculo	Resultado
Registros origen	-	13,507
Menos filas vacías	13,507 - 4,160	9,347
Menos duplicados	9,347 - 11	9,336
Registros en hechos	COUNT(fact_credito)	9,336
Diferencia	Esperado - cargado	0

8. Dashboard web y consumo mediante APIs

El dashboard principal se implementó con Flask. A diferencia de Streamlit, esta arquitectura separa explícitamente backend y frontend, pero ambos se ejecutan dentro del mismo proyecto y con un solo proceso Python.

8.1 Backend Flask

Endpoint	Metodo	Funcion
/	GET	Entrega index.html
/api/health	GET	Verifica conexión con MySQL
/api/filtros	GET	Lista opciones de filtros
/api/dashboard	GET	Devuelve KPIs, gráficos y detalle

8.2 Frontend

index.html define la estructura; styles.css controla el diseño y la adaptación responsive; dashboard.js utiliza fetch para consumir las APIs y actualiza tarjetas, gráficos Chart.js y la tabla.

```
const response = await fetch(`/api/dashboard?${params.toString()}`);
const data = await response.json();
updateKpis(data.kpis);
updateCharts(data);
updateTable(data.detalle);
```



Figura 6. Dashboard conectado al DataMart con filtros interactivos.

9. Resultados de negocio y KPIs

KPI	Resultado
Cantidad de créditos	9,336
Monto total colocado	S/ 1,695,883,888.67
Monto promedio	S/ 181,649.95
Tasa promedio	10.03%
Tasa mínima / máxima	5.9900% / 23.8900%
Plazo mínimo / máximo	14 / 300 meses

9.1 Productos

Producto	Creditos	Monto total
NMIV	8,565	S/ 1,557,092,996.98
S-CRC	352	S/ 121,318,382.44
FCTP	419	S/ 17,472,509.25

9.2 Principales departamentos

Departamento	Creditos	Monto total
LIMA	5,789	S/ 1,236,814,750.52
LAMBAYEQUE	627	S/ 80,598,442.85
CALLAO	396	S/ 79,108,207.54
PIURA	633	S/ 68,120,292.91
LA LIBERTAD	474	S/ 66,845,277.69

9.3 Hallazgos

- SEPTIEMBRE registro el mayor monto mensual: S/ 227,511,659.94.
- NMIV concentro la mayor cantidad y monto de colocaciones.
- Lima fue el departamento con mayor concentracion de creditos.
- La tasa promedio general fue 10.03% y el plazo maximo fue 300 meses.

10. Evidencias tecnicas

Evidencia solicitada	Archivo o resultado presentado
Scripts SQL	sql/00 al sql/04
Proceso ETL	etl/extract.py, transform.py, load.py y main.py
Tablas cargadas	9,336 hechos y cinco dimensiones
Carga incremental	Segunda ejecucion: 0 filas nuevas
Resultados de consultas	03_validaciones.sql y 04_consultas_kpi.sql
Pruebas	5 pruebas automatizadas aprobadas
Dashboard	Flask + HTML/CSS/JS en localhost:5000

10.1 Correspondencia con los entregables

Entregable	Estado	Evidencia
Base de datos implementada	Cumplido	MySQL mivivienda_dm
Scripts SQL	Cumplido	DDL, vistas, validaciones y KPIs
ETL funcional	Cumplido	Carga inicial e incremental
Hechos y dimensiones	Cumplido	Modelo estrella fisico
Evidencias de carga	Cumplido	Figuras 3 y 4
Validaciones realizadas	Cumplido	SQL y unittest

11. Conclusiones

- La tabla plana del archivo origen fue transformada en un modelo estrella con cinco dimensiones y una tabla de hechos.
- El proceso ETL resolvió filas vacías, formatos de fecha, normalización de texto, conversión numérica y duplicados.
- La clave record_hash permite realizar cargas incrementales sin repetir registros.
- Las pruebas confirmaron que no existen claves foráneas nulas, montos inválidos ni diferencias entre registros esperados y cargados.
- El dashboard web demuestra la explotación del DataMart mediante APIs, filtros y visualizaciones interactivas.
- La implementación satisface los entregables técnicos solicitados para el Avance 2.

Referencias

- Fondo MIVIVIENDA S.A. (2025). Colocaciones de los Créditos Mivivienda 2024. Plataforma Nacional de Datos Abiertos.

- Kimball, R., & Ross, M. (2013). The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling (3rd ed.). Wiley.
- Pandas Development Team. (2026). pandas documentation.
- Pallets Projects. (2026). Flask documentation.
- Oracle. (2026). MySQL Reference Manual.
- Universidad Tecnológica del Perú. (2026). Guía de Avance 2: Implementación del ETL y construcción del DataMart.

Anexo A. Explicación de archivos Python

Archivo	Responsabilidad
etl/conexion.py	Lee .env y crea conexiones MySQL con SQLAlchemy.
etl/setup_database.py	Crea la base y ejecuta los scripts DDL.
etl/extract.py	Lee el CSV y valida sus 14 columnas.
etl/transform.py	Limpia, convierte, valida, deduplica y genera hash.
etl/load.py	Carga staging, dimensiones, hechos y auditoría.
etl/main.py	Orquesta el ETL inicial o incremental.
web/app.py	Controlador Flask y endpoints REST.
web/services/dashboard_service.py	Consultas y lógica del dashboard.
tests/test_transform.py	Pruebas de reglas de transformación.
tests/test_web_api.py	Pruebas de página, conexión, APIs y filtros.

Anexo B. Comandos de ejecución

```

.\.venv\Scripts\python.exe -m etl.setup_database
.\.venv\Scripts\python.exe -m etl.main --mode initial
.\.venv\Scripts\python.exe -m etl.main --mode incremental
.\.venv\Scripts\python.exe -m unittest discover -s tests -v
.\.venv\Scripts\python.exe -m web.app

```

Dashboard: <http://localhost:5000>